

Кодирование алгоритма персептрона

Теперь, когда мы разработали алгоритм персептрона для своего приложения анализа настроений, создадим для него код. Сначала напишем код с нуля, чтобы он соответствовал исходному набору данных, а затем воспользуемся Tugl Create. В реальной жизни мы всегда задействуем библиотеку, и нам не нужно кодировать собственные алгоритмы. Тем не менее полезно хотя бы один раз закодировать часть алгоритмов — думайте об этом как о выполнении деления столбиком. Обычно мы делим столбиком без калькулятора — хорошо, что нам приходилось заниматься этим в школе. Теперь, проделывая это с помощью калькулятора, мы знаем, что творится «под капотом». Далее приведен код для этого раздела, а набор данных показан в табл. 5.5.

- **Блокнот** `Coding_perceptron_algorithm.ipynb`:
https://github.com/luisguiserrano/manning/blob/master/Chapter_5_Perceptron_Algorithm/Coding_perceptron_algorithm.ipynb

Таблица 5.5. Набор данных об инопланетянах, время, когда они произносили каждое из слов, и их настроение

Аак	Бип	Радостный/грустный
1	0	0
0	2	0
1	1	0
1	2	0
1	3	1
2	2	1
2	3	1
3	2	1

Начнем с определения набора данных как массива NumPy. Признаки соответствуют двум числам, соотносящимся с появлениями «аак» и «бип». Метки равны 1 для радостных предложений и 0 — для грустных:

```
import numpy as np
features = np.array([[1,0],[0,2],[1,1],[1,2],[1,3],[2,2],[2,3],[3,2]])
labels = np.array([0,0,0,0,1,1,1,1])
```

Это дает нам график, изображенный на рис. 5.25. На нем радостные предложения представлены треугольниками, а грустные — квадратами.

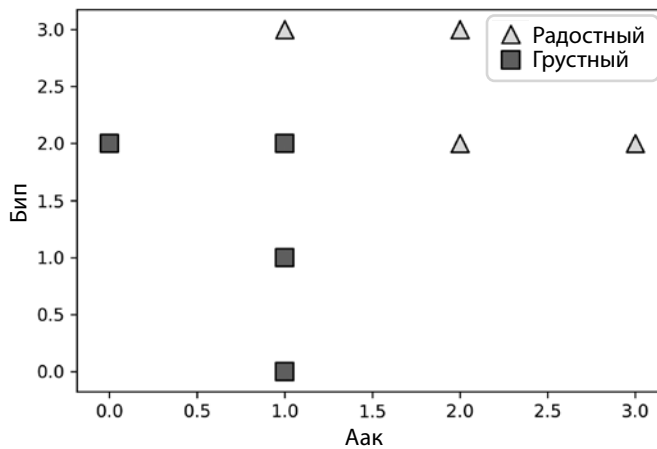


Рис. 5.25. График набора данных

Кодирование метода персептрона

В этом разделе мы закодируем метод персептрона. Сделаем это с помощью стохастического градиентного спуска (по одной точке за раз), но можем использовать также мини-пакетный или пакетный градиентный спуск. Начнем с кодирования функций оценки и прогноза. Обе они получают одни и те же входные данные, которые представляют собой веса модели, смещение и признаки одной точки данных. Функция оценки возвращает оценку, которую модель присваивает этой точке данных, а функция прогнозирования возвращает 1, если оценка больше или равна нулю, и 0, если оценка меньше нуля. Для этой функции мы используем точечное произведение, определенное в разделе «Построение графика функции ошибок и умение вовремя остановиться» главы 3:

```
def score(weights, bias, features):
    return np.dot(features, weights) + bias
```

Вычисляет точечное произведение между весами и признаками, добавляет смещение и применяет ступенчатую функцию

Чтобы создать функцию прогнозирования, сначала пишем ступенчатую функцию. Прогнозирование — это пошаговая функция оценки:

```
def step(x):
    if x >= 0:
        return 1
    else:
        return 0

def prediction(weights, bias, features):
    return step(score(weights, bias, features))
```

Смотрит на результат и, если он положительный или нулевой, возвращает 1, если отрицательный — 0

Далее мы кодируем функцию ошибки для одной точки. Напомним, что ошибка равна нулю, если точка классифицирована правильно, и абсолютному значению оценки, если неправильно. Функция принимает в качестве входных данных веса и смещения модели, а также признаки и метку точки данных:

```
def error(weights, bias, features, label):
    pred = prediction(weights, bias, features)
    if pred == label:
        return 0
    else:
        return np.abs(score(weights, bias, features))
```

Если прогноз соответствует метке, то точка хорошо классифицирована, что означает нулевую ошибку

Если прогноз отличается от метки, то точка классифицируется неправильно, а это означает, что ошибка равна абсолютному значению оценки

Теперь напомним функцию для средней ошибки перцептрона. Она вычисляет среднее значение ошибок всех точек в нашем наборе данных:

```
def mean_perceptron_error(weights, bias, features, labels):
    total_error = 0
    for i in range(len(features)):
        total_error += error(weights, bias, features[i], labels[i])
    return total_error/len(features)
```

Перебирает данные и для каждой точки добавляет ошибку в эту точку, а затем возвращает эту ошибку

Сумма ошибок делится на количество точек, и получается средняя ошибка перцептрона

Теперь, когда у нас есть функция ошибки, можем сделать следующий шаг и закодировать метод перцептрона. Мы закодируем сокращенную версию алгоритма из конца раздела «Метод перцептрона — способ немного улучшить перцептрон». Однако в записной книжке он закодирован обоими способами, первый из которых использует оператор `if` для проверки того, хорошо ли классифицирована точка:

```
def perceptron_trick(weights, bias, features, label, learning_rate = 0.01):
    pred = prediction(weights, bias, features)
    for i in range(len(weights)):
        weights[i] += (label-pred)*features[i]*learning_rate
    bias += (label-pred)*learning_rate
    return weights, bias
```

Обновляет веса и смещения с использованием метода перцептрона

Кодирование алгоритма перцептрона

Теперь, когда нам доступен метод перцептрона, можем закодировать сам алгоритм перцептрона. Напомним, что он состоит в том, чтобы, начав со случайного перцептронного классификатора, повторять метод перцептрона много раз

(по количеству периодов). Чтобы отслеживать производительность алгоритма, мы будем отслеживать среднюю ошибку персептрона в каждый из периодов. В качестве входных данных у нас имеются данные (признаки и метки), скорость обучения, которая по умолчанию равна 0,01, и количество периодов — по умолчанию 200. Код для алгоритма персептрона выглядит следующим образом:

```
def perceptron_algorithm(features, labels, learning_rate = 0.01,
    epochs = 200):
    weights = [1.0 for i in range(len(features[0]))]
    bias = 0.0
    errors = []
    for epoch in range(epochs):
        error = mean_perceptron_error(weights, bias, features, labels)
        errors.append(error)
        i = random.randint(0, len(features)-1)
        weights, bias = perceptron_trick(weights, bias, features[i],
            labels[i])
    return weights, bias, errors
```

Повторяет процесс столько раз, сколько задано периодов

Инициализирует веса равными 1, а смещение — равным 0. Вы вполне можете инициализировать их небольшими случайными числами

Массив для хранения ошибок

Вычисляет среднюю ошибку персептрона и сохраняет ее

Выбирает случайную точку в наборе данных

Применяет алгоритм персептрона для обновления весов и смещения модели на основе данной точки

Давайте же запустим алгоритм на нашем наборе данных!

```
perceptron_algorithm(features, labels)
```

Вывод: ([0.6299999999999997, 0.17999999999999938, -1.0400000000000007])

Выходные данные показывают, что полученные нами веса и смещения следующие:

- вес «аак» — 0,63;
- вес «бип» — 0,18;
- смещение — -1,04.

Мы могли бы получить и другой ответ из-за случайности выбора точек внутри алгоритма. Чтобы код в репозитории всегда возвращал один и тот же ответ, случайное начальное значение устанавливается равным нулю.

На рис. 5.26 показаны два графика: слева — подгонка прямой, справа — функция ошибок. Линия, соответствующая результирующему персептрону, выглядит как жирная прямая, которая правильно классифицирует каждую точку. Более тонкие прямые соответствуют персептронам, полученным после каждого из 200 периодов. Обратите внимание на то, как в каждом периоде прямая становится все более подходящей для точек. Ошибка уменьшается в основном по мере увеличения количества периодов, пока не достигнет нуля примерно на 140-м периоде, что означает: все точки классифицированы правильно.

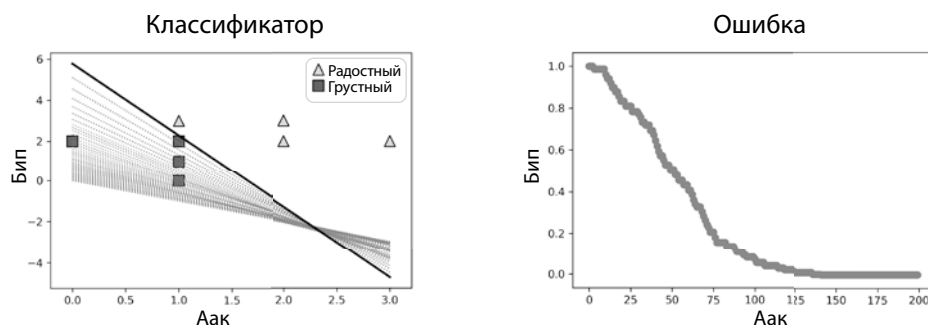


Рис. 5.26. Слева — график результирующего классификатора. Он правильно классифицирует каждую точку. Справа — график ошибок. Чем больше периодов проходит алгоритм персептрона, тем меньше становится ошибка

Это и есть код алгоритма персептрона! Как упоминалось ранее, на практике мы обычно не кодируем алгоритмы вручную, а используем библиотеки, например Turi Create или Scikit-Learn. Эту возможность и рассмотрим в следующем разделе.

Кодирование алгоритма персептрона с помощью Turi Create

В этом разделе мы научимся кодировать алгоритм персептрона в Turi Create. Код находится в том же блокноте, что и в предыдущем упражнении. Наша первая задача — импортировать Turi Create и создать SFrame со своими данными из словаря следующим образом:

```
import turicreate as tc

datadict = {'aack': features[:,0], 'beep': features[:,1], 'prediction':
labels}
data = tc.SFrame(datadict)
```

Далее мы создаем и обучаем персептронный классификатор, используя объект `logistic_classifier` и метод `create`, как показано в следующем коде. Входными данными служат набор данных и имя столбца, содержащего метки (`target`):

```
perceptron = tc.logistic_classifier.create(data, target='prediction')
```

Вывод:

Итерация	Проходы	Время	Точность обучения
1	2	1.003120	1.000000
2	3	1.004235	1.000000
3	4	1.004840	1.000000
4	5	1.005574	1.000000

УСПЕХ: Найдено оптимальное решение.